

En este proyecto estábamos analizando un PDF de un artículo como resolver problemas de geometría

Pensado durante 3m 2s >

Sí. Estábamos trabajando sobre el PDF “**FormalGeo: An Extensible Formalized Framework For Olympiad Geometric Problem Solving**”. La idea central del artículo es que los problemas de geometría olímpica pueden resolverse de forma mecanizable si primero se convierten a un **lenguaje formal geométrico**, y luego se resuelven como un problema de **búsqueda sobre teoremas y condiciones**.  2310.18021v6  2310.18021v6

El mapa del artículo era este:

1. Problema que intenta resolver

El artículo dice que la geometría formal tiene tres dificultades principales:

1. **Forma de conocimiento inconsistente**: un problema geométrico mezcla texto, diagrama, relaciones geométricas, cálculos algebraicos y razonamiento lógico.
2. **Proceso de solución poco legible**: los métodos algebraicos pueden resolver problemas, pero producen pruebas difíciles de leer.
3. **Método de solución no mecanizado**: muchas soluciones geométricas dependen de intuición humana y no de un procedimiento computacional claro.

2. Propuesta principal: GFT

Los autores proponen una teoría llamada **Geometry Formalization Theory**, o **GFT**. Esta teoría tiene tres partes:

- **Geometry ontology**: organiza los objetos geométricos, relaciones, atributos y teoremas.
- **Geometry representation theory**: explica cómo representar diagramas y razonamientos geométricos de forma formal.
- **Geometry formal language**: introduce lenguajes formales para describir problemas y teoremas.

3. FormalGeo

Sobre esa teoría construyen **FormalGeo**, un sistema formal de geometría plana. Según el artículo, FormalGeo contiene **88 predicados geométricos** y **196 teoremas**, y puede representar, validar y resolver problemas desde nivel escolar hasta nivel IMO.

Ejemplos de predicados serían:

```
Collinear (ABC)
ParallelBetweenLine (AB, CD)
IsMidpointOfLine (M, AB)
IsoscelesTriangle (ABC)
Equal (LengthOfLine (AB) , LengthOfLine (AC) )
```



La idea es que una frase como:

“M es el punto medio de AB”

se convierta en algo como:

```
IsMidpointOfLine (M, AB)
```



Y una propiedad como:

“Si M es punto medio de AB, entonces $AM = MB$ ”

se convierte en una regla formal que el sistema puede aplicar.

4. Lenguajes GDL y CDL

El artículo distingue dos lenguajes importantes:

GDL — Geometry Definition Language

Sirve para definir predicados, atributos y teoremas del sistema. Es decir, configura el “universo geométrico” que el solver conoce.

CDL — Condition Declaration Language

Sirve para escribir un problema concreto: construcción, condiciones dadas y objetivo. El artículo explica que CDL se divide en **construction statements**, **condition statements** y **goal statements**.

2310.18021v6

Por ejemplo, un objetivo podría escribirse así:

```
Relation (Parallelogram (CEGD) )
```



que significa: probar que el cuadrilátero **CEGD** es un paralelogramo.

5. Cómo “resuelve” un problema

FormalGeo no resuelve como un humano que “tiene una idea brillante” de golpe. Lo hace más como un motor de inferencia:

1. Lee las condiciones conocidas.

2. Las transforma a predicados formales.
3. Aplica teoremas permitidos.
4. Genera nuevas condiciones.
5. Repite hasta llegar al objetivo.

El artículo representa el proceso como un **hiperárbol**: los nodos son condiciones y las aristas son teoremas. Resolver el problema equivale a encontrar un camino desde las condiciones iniciales hasta el objetivo.  2310.18021v6

6. FGPS: el solver

El artículo también presenta **FGPS**, un solver escrito en Python. Su arquitectura tiene módulos como:

- control principal,
- parser de lenguaje formal,
- motor de ejecución GPL,
- solucionador de ecuaciones,
- almacenamiento de condiciones,
- interfaz para IA.

FGPS puede trabajar en modo interactivo o automático, y usa búsqueda hacia adelante, búsqueda hacia atrás y búsqueda asistida por IA.  2310.18021v6

7. Forward search y backward search

El paper compara dos formas de búsqueda:

Forward search

Parte de las condiciones conocidas y aplica teoremas hasta llegar al objetivo.

Ejemplo conceptual:

```
Dado:
IsMidpointOfLine (M, AB)

Aplicar teorema:
midpoint_of_line_property

Obtener:
Equal (LengthOfLine (AM) , LengthOfLine (MB) )
```



Backward search

Parte del objetivo y pregunta: “¿Qué condiciones bastarían para probar esto?”. Luego intenta probar esas subcondiciones.

Ejemplo conceptual:

Objetivo:
Parallelogram (ABCD)



Una forma de probarlo:
ParallelBetweenLine (AB, CD)
ParallelBetweenLine (BC, AD)

Entonces el solver reduce el problema a probar esas dos condiciones.

8. Resultados experimentales

Los autores evaluaron el sistema sobre **formalgeo7k**, un conjunto de **6,981 problemas**, expandido por aumento de datos a **133,818 problemas**. También mencionan **formalgeo-imo**, con problemas más difíciles de nivel IMO.  2310.18021v6

En los experimentos, el mejor resultado automático reportado en formalgeo7k fue con **forward random search**, alcanzando aproximadamente **39.7%** de éxito. El artículo aclara que muchos fallos se debieron a timeouts, no necesariamente a imposibilidad lógica del sistema.  2310.18021v6

Resumen en una frase

El artículo propone convertir la geometría olímpica en un sistema formal de predicados, teoremas y búsqueda, de modo que una computadora pueda **leer, verificar y producir** soluciones geométricas trazables.

Podemos continuar desde la parte más útil: traducir un problema geométrico real a **CDL**, explicar el solver **FGPS**, o reconstruir paso a paso el ejemplo del paralelogramo  CEGD.

Teniendo en cuenta este trabajo, lo que quiero hacer es crear un modelo de IA que pueda visualizar una imagen geométrica y describa en los predicados que ellos proponen para ello necesitamos que tengamos claro, como se trabaja en el artículo

Pensado durante 4m 53s >

Exacto. Para tu proyecto, el artículo debe entenderse como una arquitectura **imagen/texto** → **lenguaje formal geométrico** → **verificación/razonamiento**. Tu modelo de IA no debería intentar “resolver” desde la imagen en la primera etapa, sino **traducir el diagrama a predicados FormalGeo**, principalmente en formato **CDL**.

La idea central es esta:

```
Imagen geométrica
  ↓
Detección visual de puntos, líneas, círculos, marcas y relaciones
  ↓
Predicados FormalGeo
  ↓
construction_cdl + condition_cdl
  ↓
FGPS valida, expande condiciones y eventualmente resuelve
```



El propio artículo dice que la IA puede actuar como **parser**, permitiendo la autoformalización de lenguaje natural y diagramas geométricos; también puede actuar como solver o podador de búsqueda. FormalGeo, según el paper, contiene **88 predicados geométricos y 196 teoremas**, y FGPS es el solver que interpreta estas representaciones.  2310.18021v6

1. Qué parte del artículo nos interesa para tu modelo

Para tu objetivo, la parte más importante no es todavía el motor de búsqueda ni la demostración final. Lo más importante es entender cómo FormalGeo representa un problema.

FormalGeo separa el sistema en:

```
GDL = Geometry Definition Language
CDL = Condition Declaration Language
```



GDL define el vocabulario del sistema: qué predicados existen, qué argumentos aceptan, qué condiciones de validez tienen y qué expansiones automáticas producen.

CDL describe un problema concreto. Es decir, ahí va lo que queremos que produzca tu modelo a partir de la imagen. El artículo divide CDL en tres partes: **construction statements**, **condition statements** y **goal statements**. Las construction statements describen la estructura topológica del diagrama, por ejemplo `Shape`, `Collinear` y `Cocircular`; las condition statements describen condiciones dadas, incluyendo relaciones geométricas y algebraicas; y las goal statements declaran el objetivo del problema.  2310.18021v6

Por tanto, tu modelo debería producir algo como:

```
construction_cdl:
Shape (ABC)
Collinear (AMB)
```



```
Cocircular (O,ABC)
```

```
condition_cdl:  
Equal (LengthOfLine (AM) , LengthOfLine (MB) )  
PerpendicularBetweenLine (AC,BC)  
ParallelBetweenLine (AB,DE)
```

```
goal_cdl:  
Relation (Parallelogram (ABCD) )
```

Pero hay un matiz importante: **desde una imagen sola normalmente no se debe inferir el objetivo**. El objetivo suele venir del enunciado textual, no del dibujo.

2. La distinción clave: TSI y MI

El paper distingue dos tipos de información en un diagrama geométrico:

```
TSI = Topological Structure Information  
MI  = Metric Information
```



La **TSI** es la estructura básica del dibujo: qué puntos existen, qué puntos están conectados, qué puntos son colineales, qué figuras cerradas hay, qué puntos están sobre un círculo, qué líneas se intersectan, etc.

La **MI** es la información métrica: igualdad de longitudes, medidas de ángulos, perpendicularidad, paralelismo, radios, diámetros, tangencias, bisectrices, marcas de igualdad, marcas de ángulo recto, etc. El artículo afirma que el reto principal es formalizar la información topológica implícita en los diagramas, y clasifica la información del diagrama en TSI y MI.  2310.18021v6

Para tu modelo, esto implica dos tareas distintas:

```
Tarea visual 1: extraer TSI  
Imagen → Shape / Collinear / Cocircular
```



```
Tarea visual 2: extraer MI  
Imagen → ParallelBetweenLine / PerpendicularBetweenLine / Equal(...) / IsMidpoint..
```

La primera es más estructural. La segunda depende mucho de marcas visuales explícitas.

3. Qué debe producir el modelo primero: construction_cdl

En FormalGeo, los predicados estructurales fundamentales son:

```
Shape (...)  
Collinear (...)  
Cocircular (...)
```



El artículo dice que FormalGeo usa estos tres predicados estructurales fundamentales y que, durante la inicialización, el solver aplica el método de construcción topológica para expandir automáticamente a entidades básicas como:

```
Point
Line
Angle
Polygon
Arc
Circle
```



Es decir, tu modelo no necesita emitir explícitamente todos los `Point (A)`, `Line (AB)` o `Angle (ABC)` si ya produce bien `Shape`, `Collinear` y `Cocircular`; FGPS puede expandir esas entidades.  2310.18021v6

Ejemplo simple:

Imagen:

Triángulo ABC con M sobre AB



Salida esperada:

```
construction_cdl:
Shape (ABC)
Collinear (AMB)
```



A partir de eso, FGPS puede reconocer entidades como:

```
Point (A)
Point (B)
Point (C)
Point (M)
Line (AB)
Line (AC)
Line (BC)
Angle (ABC)
Polygon (ABC)
```



La idea es que `construction_cdl` no describe “teoremas” ni “hechos deducidos”, sino la estructura básica del dibujo.

4. Cómo FormalGeo entiende una figura cerrada

El artículo propone el **topological mapping method**. La intuición es que una figura cerrada puede representarse por una lista ordenada de puntos. Por ejemplo, un triángulo

puede verse como:

```
A → B → C → A
```



Formalmente se puede representar como:

```
Shape (ABC)
```



o, en algunas notaciones de ejemplo del artículo:

```
Shape (AB, BC, CA)
```



Para figuras compuestas, el artículo propone el **topological construction method**, que descompone el diagrama en figuras simples y luego reconstruye el diagrama compuesto preservando la TSI y la MI. [2310.18021v6](#)

Esto es muy importante para tu modelo: no tiene que reconocer “toda la figura” de golpe. Puede reconocer componentes simples.

Por ejemplo, una figura compleja puede descomponerse en:

```
Shape (ABC)
Shape (ACD)
Shape (ADE)
Collinear (BCD)
```



Y luego agregar condiciones métricas:

```
PerpendicularBetweenLine (AC, DC)
ParallelBetweenLine (AE, BD)
ParallelBetweenLine (BA, DE)
```



Ese enfoque coincide con el ejemplo del artículo sobre construcción topológica, donde se separa la información estructural de la información métrica. [2310.18021v6](#)

5. Qué debe producir después: `condition_cdl`

`condition_cdl` contiene hechos geométricos dados. En tu proyecto, estos hechos pueden venir de dos fuentes:

1. Del diagrama visual
2. Del enunciado textual



Desde la imagen se pueden extraer, por ejemplo:

Marcas de igualdad de segmentos

Si el diagrama muestra marcas iguales sobre AM y MB :

```
Equal (LengthOfLine (AM) , LengthOfLine (MB) )
```



Si además A , M , B son colineales:

```
Collinear (AMB)
```



Entonces FGPS puede llegar a:

```
IsMidpointOfLine (M, AB)
```



Pero conviene distinguir entre **observado** y **deducido**. Si la imagen solo muestra dos segmentos con marcas iguales, la salida visual más prudente es:

```
Collinear (AMB)
```

```
Equal (LengthOfLine (AM) , LengthOfLine (MB) )
```



Si el texto dice “M es punto medio de AB”, entonces sí se puede emitir directamente:

```
IsMidpointOfLine (M, AB)
```



FormalGeo define ese predicado con chequeos de existencia como $\text{Point}(M)$, $\text{Line}(AB)$ y $\text{Collinear}(AMB)$, y además puede extenderlo a una igualdad de longitudes.

 2310.18021v6

Ángulo recto

Si aparece el cuadradito de ángulo recto en ABC :

```
PerpendicularBetweenLine (BA, BC)
```



o equivalentemente:

```
Equal (MeasureOfAngle (ABC) , 90)
```



Pero para FormalGeo normalmente es más natural emitir:

```
PerpendicularBetweenLine (BA, BC)
```



Paralelismo

Si hay flechas de paralelismo en dos segmentos:

```
ParallelBetweenLine (AB,CD)
```



Igualdad de ángulos

Si hay arcos iguales en dos ángulos:

```
Equal (MeasureOfAngle (ABC) ,MeasureOfAngle (DEF) )
```



Círculo

Si hay un círculo con centro  que pasa por , , :

```
construction_cdl:  
Cocircular (O,ABC)  
  
condition_cdl:  
IsCentreOfCircle (O,O)
```



La notación `IsCentreOfCircle (O,O)` parece extraña al principio, pero aparece así en el ejemplo del paper: el primer  es el punto centro y el segundo  representa el círculo nombrado por su centro.  2310.18021v6

6. No todo lo que “parece” cierto en el dibujo debe convertirse en predicado

Esto es crucial.

En problemas olímpicos, los diagramas **no siempre están a escala**. Si dos líneas “parecen paralelas”, eso no siempre significa que debas producir:

```
ParallelBetweenLine (AB,CD)
```



Solo deberías emitirlo si:

1. Hay marca visual explícita de paralelismo;
2. El texto lo dice;
3. O el dataset asume que ciertas propiedades visuales son intencionales.



Lo mismo para:

```
Equal (LengthOfLine (AB) ,LengthOfLine (CD) )  
PerpendicularBetweenLine (AB,CD)
```



```
IsoscelesTriangle(ABC)
Parallelogram(ABCD)
```

Un buen parser geométrico debe separar:

Hechos visibles estructurales:
Shape, Collinear, Cocircular

Hechos métricos explícitos:
marcas de igualdad, ángulo recto, paralelismo, tangencia, etc.

Hechos deducidos:
lo que debe obtener FGPS mediante teoremas

El artículo justamente diseña FGPS para validar condiciones, construir diagramas automáticamente, expandir condiciones y ejecutar razonamiento algebraico/relacional.

 2310.18021v6

7. La arquitectura recomendable para tu modelo

Yo plantearía tu sistema en cinco módulos.

Módulo 1: detector visual geométrico

Entrada:

Imagen del problema

Salida intermedia:

```
<> JSON

{
  "points": ["A", "B", "C", "M", "O"],
  "segments": [["A", "B"], ["B", "C"], ["C", "A"]],
  "circles": [{"center": "O", "points": ["A", "B", "C"]}],
  "collinear_groups": [["A", "M", "B"]],
  "marks": {
    "equal_segments": [["A", "M"], ["M", "B"]],
    "right_angles": [["A", "C", "B"]],
    "parallel_groups": [[["A", "B"], ["C", "D"]]]
  }
}
```

Este módulo detecta puntos, etiquetas, líneas, arcos, círculos y marcas.

Módulo 2: constructor topológico

Convierte la salida visual en estructura FormalGeo:

```
Shape (ABC)
Collinear (AMB)
Cocircular (O,ABC)
```



Este módulo debe implementar reglas de ordenamiento: por ejemplo, si detecta un triángulo con vértices **A**, **B**, **C**, debe elegir una representación canónica para no producir duplicados como:

```
Shape (ABC)
Shape (BCA)
Shape (CAB)
```



FormalGeo contempla que una misma entidad puede tener múltiples representaciones, por ejemplo `LengthOfLine (AB)` y `LengthOfLine (BA)`, así que conviene tener una capa de normalización/canonicalización.  2310.18021v6

Módulo 3: extractor de condiciones métricas

Convierte marcas visuales en `condition_cdl`:

```
Equal (LengthOfLine (AM) , LengthOfLine (MB) )
PerpendicularBetweenLine (AC,BC)
ParallelBetweenLine (AB,DE)
Equal (MeasureOfAngle (ABC) , MeasureOfAngle (DEF) )
```



Este módulo no debe deducir teoremas; solo debe traducir lo observado.

Módulo 4: validador simbólico con FGPS

La salida del modelo debe pasar por FGPS.

FGPS tiene módulos para parser GDL/CDL, inverse parser, almacenamiento de condiciones, ejecución GPL, solver de ecuaciones e interfaz de IA. El artículo indica que el parser convierte lenguaje formal a lenguaje de máquina y que el módulo Problem asegura corrección y consistencia de las condiciones de entrada mediante construcción automática del diagrama, expansión de condiciones y chequeos de validez.

 2310.18021v6

Esto permite usar FGPS como corrector:

```
Modelo visual produce CDL
    ↓
FGPS intenta parsear
    ↓
Si falla:
    hay error sintáctico, entidad inexistente o condición inválida
```



Si pasa:
se guarda como representación formal candidata

Módulo 5: fusión imagen + texto

En problemas reales, la imagen sola no basta. El artículo dice que un problema típico contiene condiciones en lenguaje natural, objetivo y diagrama geométrico; la GFT busca transformar problemas descritos en lenguaje natural e imágenes en lenguaje formal unificado.  2310.18021v6

Entonces, idealmente tu sistema debería tener dos parsers:

```
Parser visual:
imagen → construction_cdl + condiciones visibles

Parser textual:
enunciado → condition_cdl + goal_cdl

Fusión:
construction_cdl + condition_cdl + goal_cdl
```



Ejemplo:

Texto:

```
ABC es isósceles, F es punto medio de OB, O es centro del círculo O, OG = GD.
Probar que CEGD es un paralelogramo.
```



CDL del paper:

```
condition_cdl:
IsMidpointOfLine(F,OB)
IsCentreOfCircle(O,O)
IsoscelesTriangle(ABC)
Equal(LengthOfLine(OG),LengthOfLine(GD))

goal_cdl:
Relation(Parallelogram(CEGD))
```



Y el diagrama aporta:

```
construction_cdl:
Shape(CE,ODE,DC)
Shape(ODE,EG,GD)
Shape(EA,AO,OG,GE)
Shape(GO,OF,FG)
Shape(DG,GF,FB,BD)
```



```
Shape (OBD, DB)
Collinear (CEA)
Collinear (CDB)
Collinear (AOFB)
Collinear (EGF)
Collinear (OGD)
Cocircular (O, BDE)
```

Ese ejemplo aparece explícitamente en el artículo y es una buena plantilla para tu dataset.

 2310.18021v6

8. Qué etiquetas debería aprender tu modelo

Tu modelo no debería aprender los 88 predicados de golpe. Conviene empezar con una taxonomía progresiva.

Fase 1: predicados estructurales

Estos son los más importantes para imagen:

```
Shape (...)
Collinear (...)
Cocircular (...)
```



Son la base del `construction_cdl`.

Fase 2: entidades básicas generadas o verificadas

FGPS puede expandirlas, pero internamente existen:

```
Point (A)
Line (AB)
Angle (ABC)
Polygon (ABCD)
Arc (OAB)
Circle (O)
```



En el paper aparecen como entidades básicas integradas del solver.

 2310.18021v6

Fase 3: relaciones geométricas visibles

Las más útiles para diagramas son:

```
IsMidpointOfLine (M, AB)
ParallelBetweenLine (AB, CD)
PerpendicularBetweenLine (AB, CD)
IsPerpendicularBisectorOfLine (AB, CD)
IsBisectorOfAngle (BD, ABC)
IsMedianOfTriangle (AD, ABC)
```



```
IsAltitudeOfTriangle (AD,ABC)
IsMidsegmentOfTriangle (DE,ABC)
IsDiameterOfCircle (AB,O)
IsTangentOfCircle (PA,O)
IsCentreOfCircle (P,O)
```

El artículo lista estas relaciones como parte de los predicados definidos mediante predicate definition language.  2310.18021v6

Fase 4: atributos y relaciones algebraicas

Aquí entran expresiones como:

```
LengthOfLine (AB)
MeasureOfAngle (ABC)
LengthOfArc (OAB)
MeasureOfArc (OAB)
RadiusOfCircle (O)
DiameterOfCircle (O)
AreaOfTriangle (ABC)
AreaOfQuadrilateral (ABCD)
```



Y se combinan con:

```
Equal (...)
Equation (...)
Add (...)
Sub (...)
Mul (...)
Div (...)
Sin (...)
Cos (...)
Tan (...)
```



El artículo lista estos atributos entre los predicados de FormalGeo.

 2310.18021v6

Ejemplos:

```
Equal (LengthOfLine (AB) , LengthOfLine (AC) )
Equal (MeasureOfAngle (ABC) , MeasureOfAngle (DEF) )
Equation (Sub (LengthOfLine (AB) , LengthOfLine (CD) ) )
```



9. Formato de salida recomendado para entrenamiento

Para entrenar y depurar, yo no haría que el modelo produzca solo texto plano. Usaría una salida estructurada con evidencia.

Por ejemplo:

<> JSON



```
{
  "construction_cdl": [
    "Shape (ABC) ",
    "Collinear (AMB) "
  ],
  "condition_cdl": [
    "Equal (LengthOfLine (AM) , LengthOfLine (MB) ) ",
    "PerpendicularBetweenLine (AC,BC) "
  ],
  "goal_cdl": [],
  "evidence": [
    {
      "predicate": "Collinear (AMB) ",
      "source": "line_detection",
      "confidence": 0.98
    },
    {
      "predicate": "Equal (LengthOfLine (AM) , LengthOfLine (MB) ) ",
      "source": "same_tick_mark",
      "confidence": 0.91
    },
    {
      "predicate": "PerpendicularBetweenLine (AC,BC) ",
      "source": "right_angle_marker",
      "confidence": 0.94
    }
  ]
}
```

Luego una capa simbólica lo convierte a CDL definitivo:

```
construction_cdl:
Shape (ABC)
Collinear (AMB)

condition_cdl:
Equal (LengthOfLine (AM) , LengthOfLine (MB) )
PerpendicularBetweenLine (AC,BC)
```



Esto te permite auditar errores: si FGPS falla, puedes saber si falló por OCR, detección de línea, orden de puntos, predicado inválido o inferencia indebida.

10. El principio más importante: el modelo debe ser neuro-simbólico

No conviene hacer:

imagen → LLM multimodal → CDL final



sin control.

Es mejor hacer:

imagen → detección visual → grafo geométrico → reglas simbólicas → CDL → FGPS



El modelo neuronal debe encargarse de lo difícil visual:

detectar etiquetas
detectar segmentos
detectar círculos
detectar intersecciones
detectar marcas de igualdad
detectar flechas de paralelismo
detectar ángulos rectos



La parte simbólica debe encargarse de:

ordenar puntos
normalizar predicados
evitar duplicados
verificar tipos
separar hechos observados de hechos deducidos
llamar a FGPS



Así respetas el enfoque del artículo: FormalGeo no es solo una red neuronal, sino un sistema formal verificable donde la IA se integra como parser o asistente, y el solver valida y razona sobre predicados.

2310.18021v6

11. Resumen operativo para nuestro proyecto

La forma correcta de trabajar, siguiendo el artículo, sería:

1. Definir el vocabulario FormalGeo que vamos a soportar inicialmente.
2. Empezar con Shape, Collinear, Cocircular.
3. Añadir relaciones visuales: paralelo, perpendicular, igualdad de segmentos, igual.
4. Construir un parser visual que produzca construction_cdl y condition_cdl.
5. Pasar toda salida por FGPS para validación.
6. No deducir propiedades desde la apariencia si no están marcadas o declaradas.
7. Combinar imagen + texto para obtener problemas completos.
8. Evaluar con precisión por predicado, validez CDL y éxito de FGPS.



En una frase: **tu modelo debe convertir el diagrama en una lista verificable de predicados CDL, no en una solución geométrica.** La solución debe quedar para FGPS

o para una segunda IA que opere sobre el lenguaje formal.